

2. Hafta

HTML5 ve CSS3: Sunum Katmanının İlk Malzemesi

© Öğrenme Hedefleri

Bu bölümü tamamladıktan sonra:

- Bir HTML belgesini semantik öğelerle (`header`, `nav`, `main`, `section`, `article`, `footer`), doğru başlık hiyerarşisiyle, liste, bağlantı ve resimle; ayrıca bir etkinlik listesi tablosu ve bir bilet formu iskeletiyle oluşturabileceksiniz;
- CSS seçicilerini ve özgüllük kurallarını, kaskad ve kalıtımı, kutu modelini (`content/padding/border/margin`, `box-sizing`) ve birimleri (`px/rem/%/vw`) kullanarak bir HTML belgesini biçimlendirebileceksiniz;
- Flexbox ile tek eksenli (nav çubuğu, kart satırı) ve Grid ile iki eksenli (sayfa ızgarası, koltuk planı) yerleşimler oluşturabileceksiniz;
- Duyarlı tasarım tekniklerini (viewport meta etiketi, medya sorguları, mobil-önce yaklaşım, akışkan görseller) ve erişilebilirlik temellerini (alt metin, etiket-girdi ilişkisi, kontrast, klavye odağı, landmark) uygulayabileceksiniz;
- Bu becerileri birleştirip Etkinlik/Bilet Sisteminin statik etkinlik sayfasını — nav, etkinlik kartları ızgarası, koltuk planı ızgarası, bilet formu iskeleti, mobil/masaüstü iki kırılım — oluşturabileceksiniz.

Hafta 1’de “statik içerik” ve “sunum katmanı” kavramlarını yalnız **kavram düzeyinde** gördünüz — bir sunucunun diskteki bir dosyayı olduğu gibi gönderdiğini, kullanıcının bir arayüzle etkileşime girdiğini okudunuz, ama elinizde henüz o dosyalardan biri yoktu. Bu hafta kod yazmaya başlıyorsunuz: HTML5 ve CSS3 ile, tarayıcının doğrudan okuduğu düz `.html/.css` dosyalarını bizzat yazacaksınız — derleme aracı, npm, TypeScript/React/FastAPI henüz devreye girmiyor. Yazacağınız sayfa **statik** kalacak: hiçbir JavaScript davranışı eklenmeyecek, bilet formu gönderilmeyecek, koltuk planındaki koltuklar henüz tıklanamayacak — bunların hepsi Hf.3’te TypeScript ile DOM üzerinden eklenecek. Ama HTML’iniz şimdiden **bileşene bölünebilir** biçimde kurgulanacak: her etkinlik kartı kendi başına bir `<article>`, koltuk planı kendi başına bir kapsayıcı `<div>`, form kendi başına bir `<form>` — ileride her biri ayrı bir React bileşenine dönüşecek doğal sınırlar. Dört ünite boyunca sırasıyla sayfanın **anlamını** (semantik HTML), sonra **görünümünü** (CSS temelle-

ri), sonra **yerleşimini** (Flexbox/Grid) ve son olarak **her ekrana uyumunu** (duyarlı tasarım ve erişilebilirlik) kuracaksınız.

2.1 Semantik HTML: Belge Yapısı, Form ve Tablo

1 • Ön-öğretim Neden Önemli?

Bir arama motoru botunu ya da bir ekran okuyucuyu düşünün: bu “kullanıcılar” sayfaı **göremez**, yalnızca kodu okur. Sayfanın tamamı anlamsız `<div>` kutularından oluşuyorsa, bot hangi kısmın menü, hangi kısmın asıl içerik olduğunu **anlayamaz**. Bu ünite, kodun içine “anlamı” nasıl gömdüğünüzü açar.

1 • Ön-öğretim Ön Bilgi

Hafta 1 Ünite 2’nin *statik içerik* kavramını hatırlayın: sunucunun diskte hazır duran bir dosyayı olduğu gibi gönderdiği durum. Bu ünite o dosyanın — HTML’in — içeriğini bizzat yazıyorsunuz. Hafta 1 Ünite 3’ün *sunum katmanı* kavramı da bu HTML’in tam olarak neyi temsil ettiğini hatırlatır: kullanıcının doğrudan gördüğü ve etkileşime girdiği arayüz, artık kavram olmaktan çıkıp elinizdeki gerçek bir dosyaya dönüşüyor.

Hafta 1 dört haftalık bir kavram haritası çizdi: ağ, sunucu mimarisi, WebApp’in üç katmanı, HTTP. Bu hafta o haritanın sunum katmanına ilk somut malzemeyi ekliyorsunuz. Bu ünite, o malzemenin temel iskeletini — bir HTML belgesinin nasıl başladığını, hangi parçalara bölündüğünü, hangi etiketlerin ona *anlam* kattığını — adım adım kurar.

2.1.1 Belge iskeleti

Her HTML belgesi aynı sabit iskeletle başlar. Bu iskeletin ilk satırı `<!DOCTYPE html>`’dir — tarayıcıya “bu belge güncel HTML5 standardına göre yazıldı” der; bu satır olmadan bazı eski tarayıcılar sayfayı farklı (ve tutarsız) kurallarla yorumlayabilir. Hemen ardından `<html lang="tr">` gelir: `lang` özneliği sayfanın dilini belirtir — bu, ekran okuyuculara doğru telaffuzu, arama motorlarına doğru dil filtrelemesini sağlar.

`<html>` etiketinin içi ikiye ayrılır: `<head>` ve `<body>`. `<head>` kullanıcının **görmediği** ama tarayıcının bilmesi gereken bilgileri taşır: karakter kodlamasını (`<meta charset="UTF-8">` — bu olmadan Türkçe karakterler [ğüşİöç] bozuk görünebilir), sayfanın başlığını (`<title>` — tarayıcı sekmesinde görünen metin) ve bir viewport meta etiketini:

```
1 | <meta name="viewport" content="width=device-width, initial-scale=1">
```

Bu son etiketi şimdilik olduğu gibi kabul edin; Ünite 4 duyarlı tasarımı işlerken bu etiketin **neden** zorunlu olduğunu — onsuz mobil tarayıcıların sayfayı masaüstü genişliğinde küçülterek gösterdiğini — tam olarak açacaktır. `<body>` ise kullanıcının fiilen **gördüğü** her şeyi barındırır. Aşağıdaki kesit, bu hafta oluşturacağınız etkinlik listesi sayfasının (`index.html`) tam olarak bu iskeletle başladığını gösterir:

```
1 | <!DOCTYPE html>
2 | <html lang="tr">
3 | <head>
4 |   <meta charset="UTF-8">
```

```

5 <meta name="viewport" content="width=device-width, initial-scale=1">
6 <title>KTÜ Etkinlik ve Bilet Sistemi</title>
7 <link rel="stylesheet" href="stiller.css">
8 </head>

```

Dikkat ederseniz `<head>` içinde ayrıca şu satır da vardır:

```

1 <link rel="stylesheet" href="stiller.css">

```

Bu satır, Ünite 2’de yazacağınız `stiller.css` dosyasını bu HTML belgesine bağlar; CSS kuralları HTML’e bu bağlantı sayesinde uygulanır.

Ortak Hata / Tuzak

Yeni başlayanlar bazen `<meta charset="UTF-8">` satırını atlar ya da `<head>` dışına yazar. Sonuç genelde Türkçe karakterlerin (ğ, ş, ı, ö, ç) ekranda soru işareti ya da tuhaf sembollere dönüşmesidir. Bu satırın `<head>` içinde, mümkünse ilk satırlardan biri olarak durması gerekir — tarayıcı, belgenin geri kalanını doğru kodlamayla okuyabilmek için bu bilgiye erkenden ihtiyaç duyar.

2.1.2 Div çorbasından semantik öğelere

`<body>` içindeki içeriği bölümlere ayırmanın en kestirme (ama sorunlu) yolu, her şeyi `<div>` ile sarmaktır: bir `<div>` üst menüyü, bir başkası ana içeriği, bir başkası alt bilgiyi tutar. Buna bazen **div çorbası** denir — her şey aynı, anlamsız kaba konmuştur; kodu okuyan bir insan bile hangi `<div>`’in ne işe yaradığını yalnızca `class` adına bakarak tahmin etmek zorunda kalır. `<div>`’in kendisi hiçbir anlam taşımaz — tarayıcıya “burada bir kutu var” der, ama “bu kutu menüdür” demez.

HTML5, tam bu sorunu çözmek için bir dizi **semantik HTML** etiketi tanıtır. Bu etiketlerin en önemlileri, sayfanın işlevsel bölgelerini (üst bilgi, gezinme, ana içerik, alt bilgi) işaretleyen **landmark öğeleri** (*landmark element*)’dir: `<header>` (üst bilgi/başlık alanı), `<nav>` (gezinme menüsü), `<main>` (sayfanın asıl/tekrar etmeyen içeriği, her sayfada yalnızca bir tane), `<footer>` (alt bilgi). Bunların her biri, tıpkı bir `<div>` gibi bir kutu oluşturur — ama bu kez kutunun **ne işe yaradığı** etiketin adından bellidir.

semantik HTML:
görünümünden çok anlamı taşıyan etiketlerle (ör. `<header>`, `<nav>`) yazılmış HTML (*semantic HTML*).

🔄 Analoji

Semantik HTML’i bir binadaki tabelalara benzetin: “Giriş”, “Resepsiyon”, “Ofis”, “Çıkış” tabelaları olmayan bir bina da fiziksel olarak çalışır — kapılar açılır, koridorlar oradadır — ama binaya ilk kez giren biri (ya da bir ekran okuyucu/arama motoru) yönünü bulamaz. Semantik etiketler tam olarak bu tabelalardır: binanın (sayfanın) yapısını değiştirmezler, ama o yapının hangi bölümünün ne işe yaradığını herkese (görme engelli bir kullanıcıya, bir arama motoru botuna, kodu okuyan bir geliştiriciye) açıkça duyururlar.

Aşağıdaki kesit, `index.html`’in üst kısmındaki landmark öğelerinin taslağını gösterir — `<header>` içinde bir `<nav>`, ardından `<main>`’in açılışı:

```

1 <header>
2   <nav class="site-nav">
3     <ul>
4       <li><a href="index.html">Etkinlikler</a></li>
5       <li><a href="etkinlik-detay.html">Bilet Al</a></li>
6     </ul>
7   </nav>

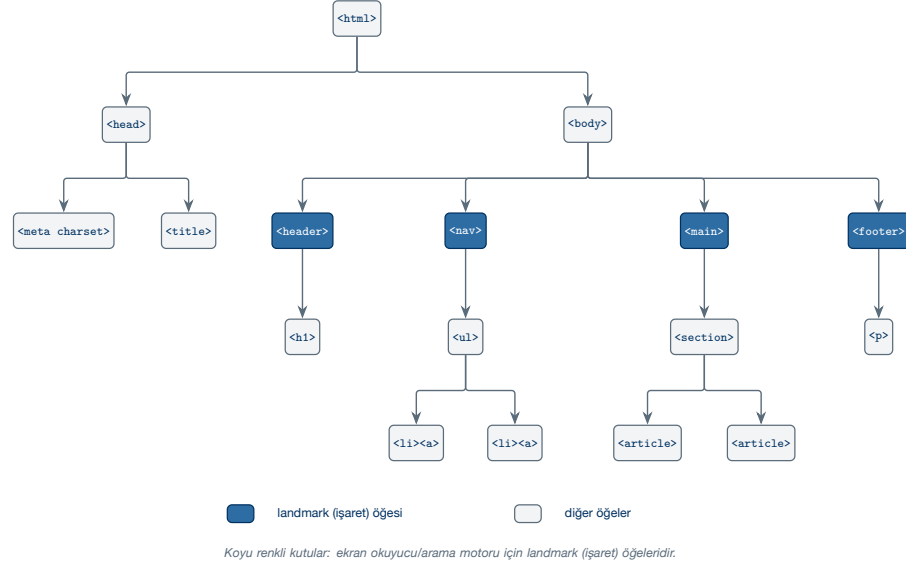
```

```

8 </header>
9
10 <main>

```

Şekil 2.1, tam bir HTML belgesinin etiketlerinin nasıl bir ağaç oluşturduğunu ve landmark öğelerinin (koyu renkli kutular) diğer öğelerden (açık gri kutular) nasıl ayrıldığını gösterir.



Şekil 2.1: Bir HTML belgesinin semantik ağaç yapısı: landmark öğeleri (koyu) ile diğer öğeler (açık) arasındaki ayrım.

`<main>` içinde, tekrar eden bir bilgi bloğu için (ör. bir etkinlik kartı) `<article>` etiketi kullanılır — kendi başına anlamlı, sayfadan koparılıp başka bir yerde de gösterilebilecek bir içerik parçasıdır. Aşağıdaki kesit tek bir etkinlik kartını gösterir:

```

1 <article class="etkinlik-karti">
2   
3   <h3>Bahar Konseri</h3>
4   <p>15 Mayıs 2026 &middot; Açık Hava Amfisi</p>
5   <p>Üniversite korosunun ve konuk sanatçıların katılımıyla bahar konseri.</p>
6   <a href="etkinlik-detay.html">Detaylar ve bilet</a>
7 </article>

```

Bu `<article class="etkinlik-karti">` yapısını bilinçli seçtiniz: her kart kendi başına bir `<article>` olduğu için, ileride (Hf.4'te) her biri doğrudan bir React bileşenine dönüşebilecek doğal bir sınırla yazılmış oluyor — bugünden itibaren HTML'inizi bu şekilde "bölünebilir" parçalar halinde kurmak, ileride kod tekrar yazmanızı gerektirmez.

Örnek Olay 2.1: Kırık başlık hiyerarşisini onarma

Aşağıdaki kesitte bir sorun var: bir `<h2>`'den sonra doğrudan bir `<h4>` kullanılmış, `<h3>` atlanmış.

```

<h1>Etkinlikler</h1>
<h2>Etkinlik Takvimi</h2>
<h4>Bahar Konseri</h4>

```

Çözüm: Başlık hiyerarşisi seviye atlamadan ilerlemelidir — bir `<h2>`'den sonra bir sonraki alt başlık her zaman `<h3>` olmalıdır, `<h4>` değil (`<h4>`, ancak o `<h3>`'ün altında kullanılabilir). Doğrusu:

```
<h1>Etkinlikler</h1>
<h2>Etkinlik Takvimi</h2>
<h3>Bahar Konseri</h3>
```

Neden önemli? Bir ekran okuyucu kullanıcısı genelde sayfada başlıktan başlığa “atlayarak” gezinir (ör. “bir sonraki h2'ye git”); seviye atlanırsa bu kullanıcı, sanki bir bölüm eksikmiş gibi bir izlenime kapılır — oysa görsel olarak bakan bir kullanıcı bu sorunu hiç fark etmeyebilir, çünkü `<h4>` de `<h3>` de kalın ve büyük görünür.

2.1.3 Başlık hiyerarşisi

başlık hiyerarşisi: h1'den h6'ya, sayfa içeriğinin önem/seviye sırasına göre dizilmesi (*heading hierarchy*).

Yukarıdaki örnek olayın dayandığı kural şudur: **başlık hiyerarşisinde** her sayfada tam olarak **bir** `<h1>` bulunur (sayfanın ana başlığı) ve alt başlıklar seviye atlamadan ilerler. Aşağıdaki kesit `index.html`'deki gerçek kullanımı gösterir:

```
1 <h1>KTÜ Etkinlik ve Bilet Sistemi</h1>
2 <p>Kampüsteki güncel etkinlikleri inceleyin, dilediğiniz etkinlik için bilet
  almaya başlayın.</p>
3
4 <h2>Öne Çıkan Etkinlikler</h2>
```

2.1.4 Listeler

Sıralı olmayan bir küme (menü öğeleri gibi, sırası önemsiz) için `<ul>` (*unordered list*), her biri bir `<li>` (*list item*) olmak üzere kullanılır — Ünite 1'in başında gördüğünüz `<nav>` içindeki menü tam olarak bu yapıdadır. Sırası **önemli** bir dizi (ör. bir işlemin adımları) için ise `<ol>` (*ordered list*) kullanılır; tarayıcı öğeleri otomatik olarak 1, 2, 3 diye numaralar. Bilet alma adımlarını sıralı bir liste olarak yazdığınızda tarayıcı şu görünümü üretir:

```
1 <ol>
2   <li>Koltuk listesinden müsait bir koltuk seçin.</li>
3   <li>Bilet formunu doldurun.</li>
4   <li>Formu gönderin ve bitirin.</li>
5 </ol>
```

İpucu

“Hangi liste türünü kullanmalıyım?” sorusunun kestirme yanıtı şudur: sırayı değiştirdiğinizde anlam bozuluyorsa `<ol>`, bozulmuyorsa `<ul>`. Bir menüde “Etkinlikler” ile “Bilet Al” öğelerinin yeri değişse anlam bozulmaz (`<ul>`); bilet alma adımlarında 2. adımı 1. adımdan önce yapmak anlamsız olur (`<ol>`).

2.1.5 Bağlantılar

Bir sayfayı bir başkasına bağlamak için `<a href="...">` etiketi kullanılır. `href` değeri iki biçimde olabilir: **görelî** (*relative*) bir yol, ör. `href="etkinlik-detay.html"` — aynı klasördeki bir dosyayı işaret eder, sunucu/klasör değişse de çalışmaya devam eder; ya da **mutlak** (*absolute*) bir yol, ör. `href="https://www.ktu.edu.tr"` —

tam adresi belirtir, başka bir sitedeki sayfaya gitmek için zorunludur. Sitenizin kendi sayfaları arasında gezinirken (ör. `index.html`'den `etkinlik-detay.html`'e) göreli yol tercih edilir; dış bir siteye bağlanırken mutlak yol kullanılır. `index.html`'in altbilgisinde gördüğünüz `target="_blank"` özniteliği, bağlantının **yeni bir sekmede** açılmasını sağlar — genelde dış sitelere giden bağlantılarda kullanılır, kullanıcı sizin sitenizden tamamen ayrılmasın diye.

2.1.6 Resimler

alt metni:
bir resmin yerini tutan,
ekran okuyucuların
okuduğu kısa açıklama
(*alt text*).

Bir resim eklemek için `` kullanılır. `src` resmin dosya yolunu (ya da URL'sini) belirtir; `alt` ise **alt metnidir**. `alt` özniteliğinin iki ayrı işlevi vardır: birincisi erişilebilirliktir — görme engelli bir kullanıcının ekran okuyucusu resmi “göremez”, bunun yerine `alt` metnini okur; ikincisi ise yedek göstergeci — resim herhangi bir nedenle (bozuk bağlantı, yavaş ağ) yüklenemezse tarayıcı resmin yerine `alt` metnini gösterir. Bu yüzden `alt` metni her zaman resmin **içeriğini** betimlemelidir, dosya adını değil: `alt="Bahar Konseri sahne görseli"`, `alt="resim1.jpg"` değil.

Ortak Hata / Tuzak

`alt=""` (boş metin) ile `alt` özniteliğini hiç yazmamak aynı şey değildir. Boş `alt=""`, ekran okuyucuya “bu resim tamamen süsleme amaçlı, atlayabilirsin” der — kasıtlı bir tercih olabilir. `alt''ı` hiç yazmamak ise ekran okuyucunun genelde dosya adını ya da tam yolu okumasına yol açar (ör. “bahar-konseri-320x180-jpg”), bu da kullanıcıyı bilgilendirmek yerine kafasını karıştırır.

2.1.7 Etkinlik listesi tablosu

Satır-sütun biçiminde, karşılaştırılabilir veriler için (ör. birden çok etkinliğin tarih/mekân/kapasite bilgisi) HTML'in tablo etiketleri kullanılır: `<table>` tüm tabloyu sarar, `<thead>` başlık satırını, `<tbody>` veri satırlarını gruplar; her satır bir `<tr>` (*table row*), başlık hücreleri `<th>` (*table header*), veri hücreleri `<td>` (*table data*) ile yazılır. Aşağıdaki kesit tablonun başlık satırını gösterir:

```
1 <table>
2   <thead>
3     <tr>
4       <th scope="col">Başlık</th>
5       <th scope="col">Tarih</th>
6       <th scope="col">Mekân</th>
7       <th scope="col">Kapasite</th>
8     </tr>
9   </thead>
```

`<th>` ile `<td>` arasındaki fark yalnızca görsel değildir: `<th>` bir **başlık** hücresi olduğunu anlamsal olarak bildirir — ekran okuyucu bir veri hücreğine geldiğinde, o hücrenin hangi sütuna ait olduğunu (“Başlık”, “Tarih” gibi) `<th>` sayesinde otomatik söyleyebilir. Tüm hücreleri `<td>` ile yazmak tabloyu görsel olarak aynı gösterse de bu bilgiyi kaybeder. Veri satırları ise `<tbody>` içinde, her etkinlik için ayrı bir `<tr>` olarak sıralanır; aşağıdaki kesit ilk satırı örnekler:

```
1 <tbody>
2   <tr>
3     <td>Bahar Konseri</td>
4     <td>15.05.2026</td>
```

```

5 <td>Açık Hava Amfisi</td>
6 <td>500</td>
7 </tr>

```

Geri kalan iki satır (“Kariyer Günleri”, “Tiyatro Gecesi”) aynı desende devam eder: her satır bir `<tr>` içinde dört `<td>` taşır; tablonun tamamı `kod_ornekleri/hafta02/index.html` dosyasındadır.

2.1.8 Bilet formu iskeleti

Kullanıcıdan veri toplayan her yapı bir `<form>` içine yazılır. Bu haftaki bilet formu, sonraki haftalarda gönderim davranışı kazanacak bir **iskelettir** — **action** özneliği henüz yoktur, “Gönder” düğmesine basıldığında sayfa aynı kalır (bu davranış Hf.3’te JavaScript ile eklenecek). Şimdilik yalnız formun **yapısına** odaklanın. Üç yapı taşına dikkat edin: birincisi, her `<input>` bir `<label for="...">` ile eşleşir: `label`’ın `for` değeri, eşleştiği `<input>`’un `id` değeriyle birebir aynıdır (ör. `for="ad-soyad" ↔ id="ad-soyad"`). Bu eşleşme yalnız görsel bir kolaylık değildir — Ünite 4’te bu eşleşmenin erişilebilirlik açısından neden önemli olduğunu göreceksiniz. Aşağıdaki kesit formun dış çerçevesini (`<fieldset>/<legend>`) ve bu eşleşmenin ilk örneğini gösterir:

```

1 <form>
2   <fieldset>
3     <legend>Bilet Bilgileri</legend>
4
5     <p>
6       <label for="ad-soyad">Ad Soyad</label>
7       <input type="text" id="ad-soyad" name="ad-soyad" required>
8     </p>

```

İkincisi, `<input>`’un `type` özneliği girdinin türünü belirler: `type="text"` düz metin, `type="date"` bir tarih seçici, `type="number"` yalnız sayı kabul eder (`min/max` ile sınırlanabilir). Aşağıdaki kesit bu iki türü örnekler:

```

1 <p>
2   <label for="etkinlik-tarihi">Tarih</label>
3   <input type="date" id="etkinlik-tarihi" name="etkinlik-tarihi" value="
4     2026-05-15">
5 </p>
6 <p>
7   <label for="bilet-adedi">Bilet Adedi</label>
8   <input type="number" id="bilet-adedi" name="bilet-adedi" min="1" max="10"
9     value="1">

```

Üçüncüsü, bir `<select>` (koltuk sınıfı seçimi gibi) sabit bir seçenek listesinden tek bir seçim yaptırır; her seçenek bir `<option>` ile yazılır. Formun geri kalanı — seçim kutusu, gönder düğmesi ve `<fieldset>`’in kapanışı — aşağıdaki kesitte:

```

1 <p>
2   <label for="koltuk-sinifi">Koltuk Sınıfı</label>
3   <select id="koltuk-sinifi" name="koltuk-sinifi">
4     <option value="ekonomi">Ekonomi</option>
5     <option value="orta">Orta Kat</option>
6     <option value="vip">VIP</option>
7   </select>
8 </p>

```

```

10 <button type="submit">Gönder</button>
11 </fieldset>
12 </form>

```

3 • Katılım Şimdi Siz Deneyin

Elinizde yalnızca `<div>` etiketleriyle yazılmış şu üç bölümlü sayfa var: bir üst menü, bir ana içerik, bir alt bilgi. Bu üç `<div>`'i uygun semantik etiketlere (`<header>`, `<main>`, `<footer>`) çevirin.

Geribildirim. Menü bölümündeki `<div>` → `<header>` içinde bir `<nav>` olur; ana içerik `<div>`'i → `<main>` olur; alt bilgi `<div>`'i → `<footer>` olur. İçerik değişmez — yalnız etiket adları değişir; ama bu küçük değişiklik, sayfayı bir ekran okuyucu ya da arama motoru için baştan sona okunabilir kılar.

Kendinizi Sınavın 2.1

Bir sayfanın tamamını `<div>` ile yazmak yerine `<header>/<nav>/<main>/<footer>` gibi semantik öğeler kullanmanın en az iki somut faydasını sayın.

5 • İzleme Aktarım — Başka Nerede?

Bu üniteye yazdığınız `<header>/<nav>/<main>/<footer>` iskeleti, Etkinlik/Bilet Sisteminin statik etkinlik sayfasının omurgasıdır. Hf.4'te her `<section>/<article>` ayrı bir React bileşenine bölünecek — bu yüzden şimdiden birbirinden net sınırlarla ayrılmış parçalar halinde yazdınız; bu ayırım, ileride kodu yeniden yazmak yerine yalnızca *taşımanızı* sağlayacak.

2.2 CSS Temelleri: Seçiciler, Kaskad, Kutu Modeli, Birimler

1 • Ön-öğretim Neden Önemli?

Bir öğeye `width: 200px` verdiniz ama ekranda beklediğinizden daha **geniş** görünüyor — neden? CSS'te bu tür “gizemli” boyut hataları, kutu modelini anlamadan sürekli karşınıza çıkar. Bu ünite bu gizemi çözer.

1 • Ön-öğretim Ön Bilgi

Ünite 1'de yazdığınız semantik HTML iskeletini ve orada tanımladığınız `class/id` özniteliklerini (ör. `class="etkinlik-karti"`) hatırlayın — bu ünite onları CSS **seçicisi** olarak kullanmaya başlar. `<link rel="stylesheet" href="stiller.css">` satırıyla HTML'e bağladığınız `stiller.css` dosyası, artık bu HTML'in **görünümünü** belirleyecek.

Ünite 1, sayfanın *yapısını* kurdu — hangi içeriğin ne anlama geldiğini. Bu ünite o yapının *görünümüne* girer: renkler, boşluklar, boyutlar, tipografi. HTML “ne” sorusuna, CSS “nasıl görünsün” sorusuna cevap verir.

2.2.1 Seçiciler

seçici: CSS kuralının hangi HTML öğelerine uygulanacağını belirleyen ifade (*selector*).

Bir CSS kuralı, HTML'in hangi öğelerine uygulanacağını bir **seçici** ile belirtir. En temel dört seçici türü şunlardır: **p** gibi bir **eleman seçici** (o etiketten sayfadaki her kopyaya uygulanır), **.etkinlik-karti** gibi bir **sınıf seçici** (başına nokta konur, aynı **class** değerine sahip tüm öğelere uygulanır — birden çok öğe aynı sınıfı paylaşabilir), **#site-basligi** gibi bir **id seçici** (başına # konur, bir sayfada yalnız bir öğede bulunması beklenen benzersiz bir kimliğe uygulanır), ve iç içe geçmiş öğeleri hedefleyen **soy seçici** — **nav ul li** gibi, aralarında boşluk bırakılan bu yazım, **<nav>** içindeki herhangi bir derinlikteki **** içindeki ****'lere uygulanır. Buna karşılık **nav > ul** gibi bir **doğrudan çocuk seçici**, yalnız **<nav>**'ın doğrudan çocuğu olan ****'a uygulanır — araya bir **<div>** girmişse bu seçici artık eşleşmez.

2.2.2 Özgüllük

özüllük: birden fazla CSS kuralı çakiştiğinde hangisinin kazanacağını belirleyen ağırlık puanı (*specificity*).

Aynı öğeye birden fazla CSS kuralı çakişirsa hangisi kazanır? Bu soruyu **özüllük** kavramı yanıtlar: id seçici, sınıf seçicisinden; sınıf seçici de eleman seçicisinden daha **ağır** sayılır (kabaca sıralama: id > sınıf > eleman). Aynı özüllükte iki kural çakişirsa, CSS dosyasında **sonradan** gelen kural kazanır.

Analoji

Özüllüğü bir öğretmenin genel kuralı ile bir öğrenciye özel notunu karşılaştırarak düşünün: öğretmenin sınıfa “herkes mavi kalemle yazsın” demesi genel bir kuraldır (eleman seçici gibi geniş kapsamlı); bir öğrenciye özel olarak “sen kırmızı kalemle yaz” demesi ise daha **özel** bir talimattır (id seçici gibi dar kapsamlı) ve genel kuralı ezer. CSS'te de daha özüllük bir seçici, genel bir kuralı ezer — sınıf ya da öğretmen kimin daha “özel” konuştuğuna bakar, kimin önce konuştuğuna değil.

Örnek Olay 2.2: İki seçicinin çakişması

stilller.css dosyasında iki farklı seçici, aynı bağlantı öğesini hedefliyor olsun:

```
nav a {
  color: mavi;
}

.site-nav a {
  color: kehribar;
}
```

<nav class="site-nav"> içindeki bir **<a>** için hangi renk uygulanır? **Adım 1:** her seçicinin özüllüğünü hesaplayın — **nav a** bir soy seçicidir, iki eleman seçicisinden oluşur (**nav + a**); **.site-nav a** ise bir sınıf seçici (**.site-nav**) ile bir eleman seçicinin (**a**) birleşimidir. **Adım 2:** sınıf seçici, eleman seçicisinden daha ağır sayıldığı için **.site-nav a** daha özüllüktür. **Sonuç:** bağlantı **kehribar** renğinde görünür — CSS dosyasındaki sıraları ne olursa olsun, çünkü özüllük farkı zaten kesin bir kazanan belirlemiştir.

2.2.3 Kaskad ve kalıtım

CSS kuralları tek bir kaynaktan gelmez: tarayıcının kendi varsayılan stilleri (ör. **<h1>**'in kalın ve büyük görünmesi), sizin yazdığınız stiller.css gibi birden çok

kaskad:

aynı özelliğe birden fazla kaynaktan gelen kuralların öncelik sırasıyla birleştirilmesi (*cascade*).

kalıtım:

bir üst öğeye uygulanan bazı CSS özelliklerinin alt öğelere otomatik geçmesi (*inheritance*).

kaynak aynı anda etkilidir. Bu kaynakların birlikte nasıl birleştirileceğini belirleyen mekanizmaya **kaskad** denir — CSS'in adındaki "C" (*Cascading Style Sheets*) tam olarak bu mekanizmadan gelir. Kaskad, çakışan kurallar arasında önce özgüllüğe, sonra kaynağın sırasına bakarak tek bir kazanan belirler; bu sayede tarayıcı hangi kuralı uygulayacağını her zaman net biçimde bilir.

Bazı CSS özellikleri, bir üst öğeye uygulandığında otomatik olarak alt öğelere de geçer; buna **kalıtım** denir. **font-family** ve **color** gibi tipografiyle ilgili özellikler tipik olarak kalıtılır — `<body>`'ye bir font ailesi verdiğinizde, aksi belirtilmedikçe içindeki **tüm** metin o fontu kullanır; siz her alt öğeye tek tek font ayarlamak zorunda kalmazsınız. Buna karşılık **border**, **padding**, **margin** gibi kutuyla ilgili özellikler kalıtılmaz — bir `<div>`'e verilen kenarlık, içindeki `<p>`'lere otomatik geçmez; her öğe kendi kutu özelliklerini ayrı ayrı tanımlamalıdır.

2.2.4 Kutu modeli

CSS'teki her HTML öğesi, aslında ekranda bir dikdörtgen — bir **kutu** — olarak yerleşir. Bu kutu tek bir parça değildir; dıştan içe doğru dört katmandan oluşan **kutu modeli** adı verilen bir yapıya sahiptir:

kutu modeli:

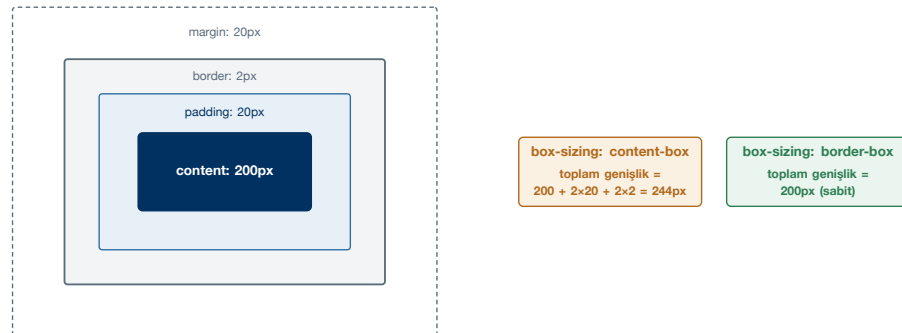
her öğenin *content* → *padding* → *border* → *margin* sırasıyla iç içe alanlardan oluşan dikdörtgen alanı (*box model*).

- **content:** gerçek içerik (metin, resim) — en içteki katman.
- **padding:** içeriğin çevresindeki iç boşluk — içerikle kenarlık arasındaki alan.
- **border:** kutunun kendi kenarlığı/çerçevesi.
- **margin:** kutunun dışındaki, komşu kutulardan bıraktığı dış boşluk — en dıştaki katman.

Analoji

Kutu modelini bir hediye kutusuna benzetin: içindeki hediye (*content*), hediyeyi saran köpük dolgu (*padding*), kutunun kendi duvarları (*border*), kutunun raftaki diğer kutulardan bıraktığı boşluk (*margin*) — dıştan içe doğru dört katman, tıpkı CSS'teki gibi.

Şekil 2.2 bu dört katmanı iç içe dikdörtgenler olarak gösterir; sağ tarafta ise bu dört katmanın **toplam genişliği** nasıl etkilendiğini karşılaştırır.



Şekil 2.2: CSS kutu modeli: content/padding/border/margin katmanları ve content-box ile border-box arasındaki toplam genişlik farkı.

Güdülemedeki gizemli boyut sorunu tam burada çözülür. Varsayılan olarak tarayıcı **content-box** modelini kullanır: **width: 200px** yazdığınızda bu değer yalnız *content* katmanına aittir; padding ve border bu genişliğin **üstüne eklenir**. Yani

`width: 200px; padding: 20px; border: 2px solid;` verilen bir kutunun ekran-daki gerçek toplam genişliği $200 + (2 \times 20) + (2 \times 2) = 244\text{px}$ olur — tam olarak güdülemedeki “beklediğimden geniş çıktı” sorunu budur.

Devreye bu problemi doğrudan çözen bir CSS özelliği girer: **box-sizing** özelliğine `border-box` değeri verildiğinde, hesaplama tersine döner: artık `width` değeri padding ve border’ı da içine alır, toplam genişlik tanımladığınız değerde **sabit** kalır — yukarıdaki örnekte kutunun toplam genişliği yine tam **200px** olur, yalnızca content alanı otomatik olarak küçülür (padding ve border’a yer açmak için). Bu yüzden `stiller.css` dosyanız en başta şu kuralla açılır:

```
1 * {
2   box-sizing: border-box;
3 }
4
5 body {
6   margin: 0;
7 }
8
9 img {
10  max-width: 100%;
11  height: auto;
12 }
```

`* { box-sizing: border-box; }` kuralı, sayfadaki her öğeye **border-box** modelini uygular — bu, gerçek projelerde neredeyse evrensel bir tercihtir, çünkü genişlik hesaplarını çok daha öngörülebilir kılar. Aynı blokta `body { margin: 0; }` tarayıcının varsayılan gövde boşluğunu sıfırlar (aksi halde sayfanın kenarında istenmeyen bir boşluk kalır), `img { max-width: 100%; height: auto; }` kuralı ise resimlerin kapsayıcılarından taşmasını engeller — bu son kuralın **neden** duyarlı tasarım için kritik olduğunu Ünite 4’te ayrıntısıyla göreceksiniz.

📷 EKLAN GÖRÜNTÜSÜ YER TUTUCU

Bir etkinlik kartı (`.etkinlik-karti`) seçiliyken tarayıcı Geliştirici Araçları’nın kutu modeli (*box model*) diyagramı — `content/padding/border/ margin` değerlerinin gerçek piksel karşılıkları görünür halde.

[assets/ss/h02-devtools-kutu-modeli.png](#)

Ortak Hata / Tuzak

Bazı öğrenciler `box-sizing: border-box`’ı yalnızca kendi yazdıkları bir öğeye verip yeterli sanır. Ama sayfadaki bir öğe **border-box**, çevresindeki başka bir öğe (ör. tarayıcı varsayılanıyla `content-box`) kullanıyorsa genişlik hesapları yine tutarsız kalır. Bu yüzden gerçek projelerde `* { box-sizing: border-box; }` gibi tüm öğeleri kapsayan bir sıfırlama kuralı, dosyanın en başına, diğer her şeyden önce yazılır.

2.2.5 Birimler

CSS’te bir uzunluk yazarken hangi **birimi** seçtiğiniz de önemlidir. **px** (piksel) bir **göreceli birim** değildir — **mutlak** bir birimdir: her zaman aynı fiziksel büyüklüğü

göreceli birim:
değeri sabit değil, kök
font/ebeveyn/ekran gibi
başka bir referansa
göre değişen ölçü birimi
(*relative unit*).

temsil eder, hangi öğeye ya da ekrana uygulanırsa uygulansın değişmez. Buna karşılık **rem**, kök öğenin (`<html>`) font boyutuna göre ölçeklenen göreceli bir birimdir — kök font 16px ise **1rem = 16px**, kullanıcı tarayıcı ayarlarından temel font boyutunu büyütürse tüm **rem** değerleri orantılı büyür. **%** bir öğenin doğrudan ebeveynine göre yüzde belirtir (ör. bir kartın genişliğini kapsayıcısının **%50'si** yapmak); **vw/vh** ise tarayıcının görünür penceresine (*viewport*) göre ölçeklenir — **100vh** her zaman ekranın tam yüksekliği kadardır, içerik ne kadar kısa olursa olsun.

Analoji

Göreceli birimi (**rem/%**) mutlak birimle (**px**) karşılaştırmak için bir tarifi düşünün: “2 su bardağı un” ile “200 gram un” arasındaki farkı. Su bardağı ölçüsü kullandığınız bardağın büyüklüğüne göre değişir — göreceli bir ölçüdür, tıpkı **rem/ %** gibi. Gram ölçüsü ise hangi kapla ölçerseniz ölçün her zaman aynı miktarı verir — mutlak bir ölçüdür, tıpkı **px** gibi.

Pratikte hangi birim ne zaman tercih edilir? Metin boyutu genelde **rem** ile yazılır (kullanıcının erişilebilirlik ayarlarına saygı gösterir); bir kapsayıcının genişliği genelde ebeveynine göre **%** ile; tam ekran yükseklik gerektiren bir bölüm ise **vh** ile tanımlanır. `stiller.css` dosyanızın renk ve tipografi bloğu bu tercihleri gösterir:

```
1 :root {
2   --renk-lacivert: #003160;
3   --renk-mavi: #2E6DA4;
4   --renk-acikmavi: #E8F0F7;
5   --renk-gri: #5A6B7A;
6   --renk-acikgri: #F2F4F6;
7   --renk-yesil: #2E7D52;
8   --renk-kehribar: #B0681B;
9 }
10
11 body {
12   font-family: "Segoe UI", Helvetica, Arial, sans-serif;
13   font-size: 1rem;
14   line-height: 1.5;
15   color: var(--renk-lacivert);
16   background-color: #FFFFFF;
17 }
```

Bu blokta ilk dikkat çeken şey **:root** seçicisidir — kök öğeyi (`<html>`) hedefler ve içine **özel özellikler** (`--renk-lacivert` gibi, başına çift tire konan CSS değişkenleri) tanımlar. Bu değişkenler sayesinde KTÜ paletindeki (`#003160` gibi bir **hex** kodu, ya da `rgb()` ile kırmızı/yeşil/mavi bileşenleri) renkler dosyanın her yerinde `var(--renk-lacivert)` şeklinde tek bir isimle çağrılır — rengi değiştirmek istediğinizde tek bir satırı güncellersiniz, dosyadaki yüzlerce kullanımı tek tek değiştirmezsiniz. **font-family** satırındaki fontlar bir **yığın** halinde sıralanır ("Segoe UI", Helvetica, Arial, sans-serif) — tarayıcı listedeki ilk fontu bulamazsa sırayla sonrakini dener; **line-height: 1.5** ise satırlar arası boşluğu ayarlayarak gövde metnini daha okunabilir kılar.

3 • Katılım Şimdi Siz Deneyin

İçeriği **200px** genişliğinde, **padding: 20px**, **border: 2px solid** olan bir kutunun toplam genişliğini **box-sizing: content-box** iken elle hesaplayın. Sonra **box-sizing: border-box** eklerseniz toplam genişlik ne olur?

Geribildirim. `content-box`'ta toplam genişlik = içerik + iki taraflı padding + iki taraflı border = $200 + (20 \times 2) + (2 \times 2) = 244\text{px}$. `border-box`'ta ise padding ve border genişliğin içine sayıldığından toplam genişlik tanımlanan değerde sabit kalır: **200px**.

Kendinizi Sınavın 2.2

Bir öğeye hem `.kart { color: mavi; }` hem `#ozel-kart { color: kirmizi; }` kuralı uygulanmışsa hangisi kazanır ve neden?

5 • İzleme Aktarım — Başka Nerede?

Bilet sistemi kartlarında tutarlı boşluk ve kenarlık hesaplaması için tüm proje `box-sizing: border-box` kullanacak — bu tercih `stilller.css`'in en başında, sıfırlama kuralında zaten yaptınız. Aynı tercih Hf.4'te React bileşen stillerinde de aynen sürecektir; bir proje boyunca kutu modeli davranışının tutarlı kalması, ileride karşılaşacağınız “beklenmedik genişlik” hatalarının büyük kısmını şimdiden önler.

2.3 Yerleşim: Flexbox ve Grid

1 • Ön-öğretim Neden Önemli?

Bir sinema ya da tiyatro koltuk planını yan yana kutularla dizmeye çalışırsanız satırlar kayabilir, koltuklar taşabilir. Bu, aslında **iki boyutlu** bir problemdir (hem satır hem sütun); tek boyutlu bir araçla (yalnızca yan yana dizmek) düzgün çözülemez. Bu ünite iki boyutlu bir problem için iki boyutlu bir araç (Grid) tanıtır.

1 • Ön-öğretim Ön Bilgi

Ünite 2'nin kutu modelini ve birimlerini (`px`, `rem`, `fr` gibi) hatırlayın — Grid ve Flexbox, kutuların içindeki boşlukları tam olarak bu birimlerle ölçer. Ayrıca Ünite 1'in `<nav>` içindeki `<ul>/<li>` listesini de hatırlayın — bu liste, varsayılan olarak dikey (alt alta) dizilir; bu ünite onu yatay bir menüye dönüştürecek araçları tanıtır.

Ünite 1 sayfanın anlamını, Ünite 2 görünümünü (renk, boşluk, boyut) kurdu. Ama şimdiye kadar hiçbir öğeyi **birbirine göre** nasıl yerleştireceğinizi görmediniz — kartlar hâlâ alt alta, koltuklar hâlâ sırayla akıyor. Bu ünite, öğeleri bilinçli olarak yan yana, satır ve sütun halinde dizmenin iki temel aracını açar.

2.3.1 Flexbox: tek eksende dizim

Flexbox: öğeleri tek bir eksen (satır ya da sütun) dizim CSS yerleşim modeli (*flexbox*).

Flexbox, bir kapsayıcıya `display: flex` verildiğinde devreye girer; kapsayıcının doğrudan çocukları artık ya bir **satırda** (`flex-direction: row`, varsayılan) ya da bir **sütunda** (`flex-direction: column`) dizilir. `justify-content` öğeleri ana eksen boyunca hizalar (ör. `center` ortalar, `space-between` aralarına eşit boşluk koyar); `align-items` ise çapraz eksen boyunca hizalar (ör. `center` öğeleri dikey ortalar, satır

yönündeyse). **flex-wrap: wrap** verilirse, kapsayıcıya sığmayan öğeler bir sonraki satıra taşar — verilmezse tek satırda sıkışmaya devam ederler.

Analoji

Flexbox'ı bir sırada yan yana dizilen insanlara benzetin: tek bir sırada (tek eksen) durup aralarına eşit boşluk bırakabilir, sıraya sıkışabilir ya da ortalana birlersiniz — ama sıra **tek yönlüdür**: ya bir satır ya bir sütunsunuz, aynı anda ikisi birden olamazsınız.

`stilller.css` dosyanızdaki nav bloğu, Ünite 1'de yazdığınız `<ul>/<li>` menüsünü bir flex kapsayıcısına çevirir:

```
1 .site-nav ul {
2   list-style: none;
3   margin: 0;
4   padding: 0;
5   display: flex;
6   flex-direction: column;
7   gap: 0.5rem;
8 }
9
10 .site-nav a {
11   text-decoration: none;
12   font-weight: bold;
13 }
```

Bu blok varsayılan (mobil) durumda **flex-direction: column** kullanır — menü öğeleri hâlâ alt alta dizilir, ama artık aralarında **gap: 0.5rem** kadar düzenli bir boşluk vardır (eskiden `<ul>`'un kendi iç boşluğuna bağlıydınız). **flex-direction**'ı **row** yapmak, menüyü tek bir satırda yan yana dizer — bu geçişin **tam olarak ne zaman** devreye gireceğini (ekran genişliği bir eşiği aştığında) Ünite 4'te göreceksiniz; şimdilik yalnız flex kapsayıcısının hem satır hem sütun yönünde çalışabildiğini bilmeniz yeterli.

Örnek Olay 2.3: Sabit sütun sayısından esnek sütun sayısına

Etkinlik kartlarınızı üç sabit sütuna (**repeat(3, 1fr)**) dizmek yerine, ekran genişliğine göre **kendiliğinden** kaç sütun sığacağına tarayıcının karar vermesini isteseydiniz ne yazardınız?

```
.etkinlik-listesi {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1.5rem;
}
```

Adım adım: **repeat(auto-fit, ...)** tarayıcıya “bu kalıbı, sığdığı kadar tekrarlar” der; **minmax(220px, 1fr)** her sütuna “en az 220px, en çok kapsayıcının eşit payı kadar geniş ol” sınırı koyar. Sonuç: geniş bir ekranda 4-5 kart yan yana sığarken dar bir ekranda otomatik olarak 1-2 sütuna iner — **hiçbir medya sorgusu yazmadan**. Bu haftaki projeniz daha öngörülebilir olsun diye sabit **repeat(3, 1fr)** + medya sorgusu yaklaşımını kullanır (Ünite 4), ama **auto-fit/minmax()** ikilisi gerçek projelerde sık görülen tamamlayıcı bir tekniktir.

2.3.2 Grid: iki eksenle yerleşim

Grid:
öğeleri satır ve sütun
olmak üzere iki eksenle
yerleştiren CSS
yerleşim modeli (*grid*).

Flexbox tek eksenle güçlüdür, ama bir tabloyu andıran, hem satırların hem sütunların birlikte tanımlı olması gereken bir yerleşimde (koltuk planı gibi) yetersiz kalır — satırlar sarıldığında sütunlar hizasını kaybedebilir. Bu boşluğu **Grid** doldurur. `display: grid` verilen bir kapsayıcıda, `grid-template-columns` sütun sayısını/genişliklerini, `grid-template-rows` satır sayısını/yüksekliklerini tanımlar; `repeat(4, 1fr)` gibi bir ifade “4 eşit genişlikte sütun” demenin kısayoludur (`fr`, kalan boşluğu paylaştıran *fraction* birimidir); `gap` ise hücreler arası boşluğu tek bir özellik ile ayarlar.

Analoji

Grid’i bir sinema salonunun koltuk planına benzetin: koltuklar hem satır hem sütun halinde düzenlidir; “3. sıra, 5. koltuk” demek için iki eksene birden ihtiyacınız vardır — yalnızca “5. sıradaki koltuk” demek yetmez, hangi **sırada** olduğunu da belirtmeniz gerekir. Grid tam olarak bunu sağlar: her hücrenin hem satır hem sütun konumu aynı anda tanımlıdır.

Etkinlik detay sayfanızdaki koltuk planı, tam olarak bu ihtiyaç için yazıldı. Önce HTML’deki 12 koltuğu (3 satır × 4 sütun) düz bir `<div>` kapsayıcısına yerleştirdiniz:

```
1 <div class="koltuk-planı">
2   <div class="koltuk">A1</div>
3   <div class="koltuk">A2</div>
4   <div class="koltuk">A3</div>
5   <div class="koltuk">A4</div>
6   <div class="koltuk">B1</div>
7   <div class="koltuk">B2</div>
8   <div class="koltuk">B3</div>
9   <div class="koltuk">B4</div>
10  <div class="koltuk">C1</div>
11  <div class="koltuk">C2</div>
12  <div class="koltuk">C3</div>
13  <div class="koltuk">C4</div>
14 </div>
```

Sonra bu kapsayıcıyı bir Grid’e çevirdiniz:

```
1 .koltuk-planı {
2   display: grid;
3   grid-template-columns: repeat(4, 1fr);
4   grid-template-rows: repeat(3, 1fr);
5   gap: 0.5rem;
6   max-width: 480px;
7   margin: 1rem 0;
8 }
9
10 .koltuk {
11   padding: 0.75rem;
12   text-align: center;
13   border: 1px solid var(--renk-yesil);
14   border-radius: 0.25rem;
15   background-color: var(--renk-acikmavi);
16 }
```

Bu iki satır dört eşit sütun ve üç eşit satır tanımlar:

```
1 grid-template-columns: repeat(4, 1fr);
```

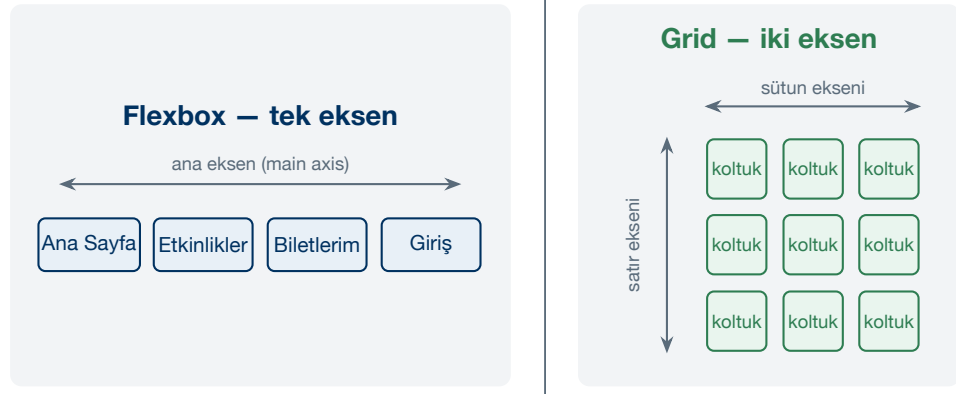
```
2 | grid-template-rows: repeat(3, 1fr);
```

12 `.koltuk` ögesi, HTML’de yazıldığı sırayla (soldan sağa, yukarıdan aşağıya) bu 3×4’lük ızgaraya otomatik yerleşir; siz hiçbir koltuğun “3. sıra, 2. sütun”da olduğunu tek tek belirtmezsiniz — Grid bunu düzen (*template*) tanımından çıkarır.

Aynı `display: grid` tekniği, kart listenizde de kullanılır — her kartın kendi başına bir `<article>` olması gibi, sayfa genelindeki ızgara da CSS Grid ile kurulur:

```
1 | .etkinlik-listesi {
2 |   display: grid;
3 |   grid-template-columns: 1fr;
4 |   gap: 1.5rem;
5 |   margin: 1rem 0;
6 | }
7 |
8 | .etkinlik-karti {
9 |   border: 1px solid var(--renk-gri);
10 |  border-radius: 0.5rem;
11 |  padding: 1rem;
12 |  background-color: var(--renk-acikgri);
13 | }
```

Şekil 2.3, Flexbox’ın tek eksenli (nav çubuğu) ile Grid’in iki eksenli (koltuk planı) yerleşimini yan yana karşılaştırır.



Flexbox: sıra ya da sütun — tek yön. Grid: sıra VE sütun — birlikte.

Şekil 2.3: Flexbox’ın tek eksenli yerleşimi ile Grid’in iki eksenli yerleşimi arasındaki fark: nav çubuğu (Flexbox) ve koltuk planı (Grid).

⚠ Dikkat

`grid-template-columns: repeat(4, 1fr)` yalnız sütun sayısını sabitler, satır sayısını değil. Koltuk sayınız 12’den 13’e çıkarsa, Grid otomatik olarak fazladan bir satır açar (13. koltuk 4. satıra düşer) — bu genelde istenen davranıştır. Ama `grid-template-rows` ile satır sayısını da sabit 3 olarak belirlediyseniz, taşan koltuklar tanımlı alanın dışına, üst üste binerek yerleşebilir. Koltuk sayısı sabit değilse `grid-template-rows` yerine `grid-auto-rows` kullanmak, satır sayısının içerik kadar esnek büyümesini sağlar.

3 • Katılım **Şimdi Siz Deneyin**

Verilen dikey `<ul><li>` menüsünü Flexbox ile yatay bir nav çubuğuna çevirin. Ardından 3 satır × 4 sütunluk bir koltuk ızgarası için gereken CSS Grid kurallarını yazın.

Geribildirim. Nav çubuğu için `.site-nav ul { display: flex; gap: 1rem; }` yeterlidir (öğeler artık yatay dizilir, varsayılan `flex-direction: row` değeriyle). Koltuk ızgarası için `.koltuk-plani { display: grid; grid-template-columns: repeat(4, 1fr); gap: 0.5rem; }` 4 sütunlu, eşit genişlikli bir ızgara oluşturur — satır sayısı içerik kadar otomatik artar.

 Kendinizi Sınavın 2.3

Satır × sütun düzenindeki bir koltuk planı için neden Flexbox yerine Grid tercih edilmelidir?

5 • İzleme **Aktarım — Başka Nerede?**

Bu ünite de oluşturduğunuz koltuk ızgarası, Hf.3'te TypeScript ile tıklanabilir hale gelecek (bir koltuğa tıklayınca rengi değişecek), Hf.5'te React'te bir dizi üzerinden `map` ile dinamik olarak üretilecek — HTML/CSS yapısı değişmeyecek, yalnız üzerine davranış eklenecek. Aynı şekilde nav çubuğunuzdaki Flexbox düzeni, sitenin her sayfasında (Hf.4'te her React sayfasında) tekrar kullanılacak ortak bir yerleşim kalıbı olarak kalacak.

2.4 Duyarlı Tasarım ve Erişilebilirlik Temelleri

1 • Ön-öğretim **Neden Önemli?**

Kullanıcıların büyük bir kısmı artık sitelere telefonda giriyor. Masaüstünde güzel görünen 3 sütunlu bir kart ızgarası, telefon ekranında yan yana sıkışıp okunmaz hale gelebilir. Bu ünite, tek bir HTML/CSS ile sayfanın her ekrana nasıl uyum sağladığını gösterir.

1 • Ön-öğretim **Ön Bilgi**

Ünite 3'ün Grid/Flexbox yerleşimini hatırlayın — kartlarınız zaten `display: grid` ile diziliydi, nav çubuğunuz zaten `display: flex` ile. Ayrıca Ünite 1'de belge iskeletine zaten eklediğiniz viewport meta etiketini hatırlayın; bu ünite ona bir de medya sorgusu **eşiği** ekler.

Ünite 3, öğeleri bir sayfada nasıl dizeceğinizi gösterdi — ama tek bir sabit düzen varsaydı. Gerçekte aynı sayfa, bir telefonda da bir masaüstünde de açılır; bu ünite aynı HTML/CSS'in **her iki** ekranda da düzgün çalışmasını sağlayan teknikleri açar.

2.4.1 Viewport meta etiketi

Ünite 1'de belge iskeletinize şu satırı zaten eklemiştiniz:

```
1 <meta name="viewport" content="width=device-width, initial-scale=1">
```

Viewport:
tarayıcının web sayfasını
gösterdiği görünür alan
(*viewport*).

Şimdi bu satırın **neden** zorunlu olduğunu açıklamanın zamanı geldi. **Viewport**, mobil bir tarayıcının sayfayı çizdiği görünür pencere genişliğidir. Bu meta etiketi olmadan mobil tarayıcılar, sayfayı — sanki hâlâ geniş bir masaüstü ekranındaymış gibi — yaklaşık 980px genişliğinde çizip sonra tamamını küçülterek gösterir; sonuç, okumak için sürekli yakınlaştırmamız gereken minicik bir sayfadır. `width=device-width` tarayıcıya “viewport genişliğini cihazın gerçek ekran genişliği yap” der; `initial-scale=1` ise başlangıç yakınlaştırma oranını %100’de sabitler. Bu iki değer olmadan, aşağıda göreceğiniz medya sorguları da anlamsız kalır — tarayıcı sayfayı zaten küçültürken gösterdiği için hiçbir eşik doğru tetiklenmez.

2.4.2 Medya sorguları

Medya sorgusu:
ekran genişliği gibi
koşullara göre farklı
CSS kuralları¹
uygulanmasını sağlayan²
yapı (*media query*).³

Medya sorgusu, CSS’e “eğer şu koşul sağlanıyorsa şu kuralları da uygula” demenin yoludur:

```
@media (min-width: 768px) {  
  /* sadece ekran 768px ve üzerinde devreye girer */  
}
```

`min-width: 768px` koşulu, “ekran genişliği en az 768 piksel ise” anlamına gelir — bu eşik altındaki ekranlarda blok içindeki kurallar hiç uygulanmaz. Bu sayı gelişigüzel değildir: 768px, tablet/masaüstü sınıfı ekranlar için yaygın kabul görmüş bir eşiktir (bu haftaki projenizde de bu değer kullanılır); farklı projelerde farklı eşikler seçilebilir.

2.4.3 Mobil-önce yaklaşım

mobil-önce yaklaşım:
önce dar ekran stilini
yazıp ekran
genişledikçe stil ekleme
stratejisi (*mobile-first
approach*).

Medya sorgularını yazarken izlenebilecek iki strateji vardır: önce geniş ekran için tasarlayıp dar ekranda özellik **çıkarmak**, ya da önce dar ekran için tasarlayıp geniş ekranda özellik **eklemek**. Bu ders — ve modern web geliştirmenin neredeyse tamamı — **mobil-önce yaklaşım**ı izler: temel (varsayılan, medya sorgusu dışındaki) kurallar en dar ekranı hedefler; `min-width` sorguları yalnızca ekran genişledikçe **ek** kurallar getirir, var olan hiçbir kuralı iptal etmez.

🔄 Analoji

Mobil-önce/medya sorgusunu esnek, ayarlanabilir bir kıyafete benzetin: önce en dar bedene (mobil) göre dikilir; beden büyüdükçe (ekran genişledikçe) belirli eşiklerde ek parça/lastik (medya sorgusuyla gelen yeni stiller) devreye girer. Kıyafet baştan geniş dikilip sonra daraltılmaz — tam tersi bir sıra, hem daha az CSS yazmanızı sağlar (çoğu mobil kullanıcı hiçbir zaman genişletilmiş kuralları indirmez), hem de “genişte güzel, darda bozuk” hatasını baştan engeller.

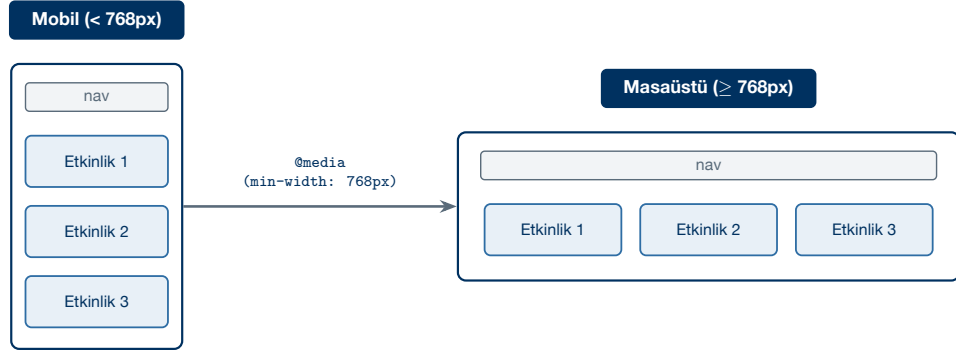
`stilller.css` dosyanızın en sonundaki blok, tam olarak bu stratejiyi uygular:

```
1 @media (min-width: 768px) {  
2   .site-nav ul {  
3     flex-direction: row;  
4     gap: 1.5rem;  
5   }  
6  
7   .etkinlik-listesi {  
8     grid-template-columns: repeat(3, 1fr);  
9   }
```

10 }

Bu blok yürürlüğe girmeden önce (yani 768px altındaki her ekranda), `.site-nav` ul zaten `flex-direction: column` (Ünite 3'te gördüğünüz taban kural) kullanır ve `.etkinlik-listesi` zaten `grid-template-columns: 1fr` (tek sütun) kullanır — bunlar **taban** (mobil) kurallardır. 768px eşiği aşıldığında bu blok devreye girer ve nav'ı `flex-direction: row`'a, kart ızgarasını `repeat(3, 1fr)`'e (üç sütuna) **günceller**. Taban kurallar silinmez, yalnızca üzerine yeni değerler yazılır — kaskadın (Ünite 2) “sonradan gelen kazanır” kuralı burada da işler.

Şekil 2.4 aynı etkinlik sayfasının mobil ve masaüstü kırılımlarını, aralarındaki eşikle birlikte gösterir.



Mobil-önce: Önce dar ekran için stil yazılır, sonra medya sorgusuyla geniş ekran stili EKLENİR.

Şekil 2.4: Aynı HTML/CSS'in mobil (tek sütun) ve masaüstü (üç sütun) kırılımları, `@media (min-width: 768px)` eşiğiyle birlikte.

📷 EKLAN GÖRÜNTÜSÜ YER TUTUCU

Tarayıcı penceresi 768px'in üzerinde bir genişliğe ayarlanmış (ör. Geliştirici Araçları'nın cihaz araç çubuğuyla ya da pencereyi elle genişleterek) hâlde `index.html` — nav çubuğunun yatay bir satıra, etkinlik kartlarının üç sütunlu bir ızgaraya dönüştüğü görünür olmalı.

`assets/ss/h02-index-masaustu-gorunum.png`

2.4.4 Akışkan görseller

Duyarlı bir sayfada dar bir ekrana geniş bir resim koyarsanız, resim kapsayıcısından taşıp yatay bir kaydırma çubuğu doğurabilir. Bunu önleyen kural **akışkan görsel** tekniğidir. Bu kuralı aslında Ünite 2'de, kutu sıfırlama bloğunun içinde zaten yazmıştınız:

akışkan görsel:
kapsayıcısına göre
otomatik küçülüp
büyüyen resim (*fluid*
image).

```
1 img {
2   max-width: 100%;
3   height: auto;
4 }
```

`max-width: 100%` resmin genişliğinin kapsayıcısının genişliğini Asla aşmamasını sağlar — resim küçükse olduğu boyutta kalır, büyükse kapsayıcıya sığacak kadar küçülür. `height: auto` ise yüksekliğin genişlikle **orantılı** kalmasını sağlar; bu olmadan yalnız `max-width` vermek resmi yamultabilir (en-boy oranı bozulabilir).

Örnek Olay 2.4: Akışkan görsel hesabı

Bir etkinlik kartınızın (`.etkinlik-karti`) genişliği 480px, içine koyduğunuz resmin doğal (orijinal) genişliği ise 800px olsun. `max-width: 100%; height: auto;` kuralı olmadan resim ekranda kaç piksel genişliğinde görünür, kuralla birlikte kaç piksel görünür?

Kural olmadan: resim kendi doğal genişliğinde (800px) çizilir — kartın 480px'lik sınırını 320px aşar, kartın dışına taşar ya da yatay kaydırma çubuğu doğurur. **Kural ile:** `max-width` değeri resmi kapsayıcısının (480px) genişliğine sıkıştırır; resim tam 480px genişliğinde görünür, `height: auto` sayesinde yüksekliği de bu yeni genişlikle orantılı olarak otomatik hesaplanır — görsel oranı bozulmadan küçülür.

2.4.5 Erişilebilirlik temelleri

Erişilebilirlik: web içeriğinin engelli kullanıcılar dahil herkes tarafından kullanılabilir olması (*accessibility*).

Erişilebilirlik, bugüne kadar öğrendiğiniz birçok tekniği tek bir amaç etrafında toplar. Ünite 1'de tanımladığınız `alt` metni, görme engelli bir kullanıcının ekran okuyucusuna resmin içeriğini duyurur; yine Ünite 1'in bilet formunda gördüğünüz `<label for="..." / <input id="...">` eşleşmesi, kullanıcının etikete tıkladığında ilgili girdinin otomatik **odaklanmasını** sağlar — bu, hem fare kullanan hem de dokunmatik ekranlı kullanıcılar için tıklanabilir alanı genişletir.

Buna ek olarak iki temel kural daha vardır. Birincisi **kontrast**: metin ile arka plan renginin arasında yeterli parlaklık farkı olmalıdır — KTÜ paletindeki koyu lacivert (`#003160`) metnin beyaz zemin üzerinde kullanılması tesadüf değildir, düşük görme keskinliğine sahip kullanıcılar için bile okunabilir bir fark bırakır. İkincisi **klavye odağı**: fare kullanamayan bir kullanıcı sayfada `Tab` tuşuyla gezinir; o an hangi öğe olduğunu görebilmesi için odaklanan öğenin etrafında görünür bir çerçeve (*focus outline*) kalmalıdır:

```
1 a:focus,
2 button:focus,
3 input:focus,
4 select:focus {
5   outline: 3px solid var(--renk-kehribar);
6 }
```

Son olarak, Ünite 1'de tanıttığınız landmark öğeleri (`<header>`, `<nav>`, `<main>`, `<footer>`) burada tekrar devreye girer: bir ekran okuyucu kullanıcısı genelde “ana içeriğe atla” gibi kısayollarla gezinir; bu atlama yalnızca sayfa gerçek landmark öğeleriyle işaretlenmişse çalışır — yani Ünite 1'de “anlam” için attığınız adım, burada doğrudan bir erişilebilirlik kazanımına dönüşür.

Ortak Hata / Tuzak

Bazı geliştiriciler, odak çerçevesini (*focus outline*) “çirkin” bulup `outline: none;` ile tamamen kaldırır. Bu, fare kullanan kişi için görsel olarak “temiz” görünse de klavye ile gezinen bir kullanıcıyı tamamen **kör** bırakır — o kullanıcı artık sayfada nerede olduğunu hiçbir şekilde göremez. Odak çerçevesi kaldırılacaksa mutlaka — görünürlüğü en az o kadar net olan — bir alternatif (ör. farklı bir `outline` rengi/kalınlığı, asla `none` tek başına) değiştirilmelidir.

3 • Katılım Şimdi Siz Deneyin

3 sütunlu bir kart ızgarasının varsayılan (mobil) durumda tek sütun olmasını, yalnızca ekran 768px'i aştığında 3 sütuna dönüşmesini sağlayan medya sorgusunu yazın.

Geribildirim. Temel (mobil) kural `.etkinlik-listesi { display: grid; grid-template-columns: 1fr; }` olur; `@media (min-width: 768px) { .etkinlik-listesi { grid-template-columns: repeat(3, 1fr); } }` eklenir — mobilde tek sütun, 768px ve üzerinde 3 sütun görünür; bu, “önce dar ekran, sonra ekle” (mobil-önce) mantığının doğrudan uygulamasıdır.

Kendinizi Sınavın 2.4

Mobil-önce yaklaşımda medya sorguları genellikle `min-width` ile mi yoksa `max-width` ile mi yazılır ve neden?

5 • İzleme Aktarım — Başka Nerede?

Bu haftanın statik etkinlik sayfası mobil ve masaüstü iki kırılımda test edilecek; Hf.14'te bu duyarlı tasarım, Docker ile canlıya alınan uygulamaya farklı cihazlardan (telefon, tablet, masaüstü) gerçek kullanıcı trafiği geldiğinde önemini koruyacak. Dört ünite boyunca kurduğunuz her parça — semantik iskelet, tutarlı kutu modeli, Flexbox/ Grid yerleşimi, duyarlı/erişilebilir davranış — şimdi tek bir çalışan sayfada bir araya geldi: Etkinlik/Bilet Sisteminin statik etkinlik sayfası artık tarayıcınızda açılıp gezinilebilir durumda.

Bölüm Özeti

Özet

Bu hafta, Hafta 1'in yalnızca kavram düzeyinde bıraktığı sunum katmanına ilk somut malzemeyi eklediniz. İlk ünite de bir HTML belgesinin iskeletini (`<!DOCTYPE html>`, `<head>`, `<body>`) ve anlamını taşıyan semantik/landmark öğelerini (`<header>`, `<nav>`, `<main>`, `<footer>`) — “div çorbası”nın aksine — yazdınız; başlık hiyerarşisi, listeler, bağlantılar, resimler, bir etkinlik tablosu ve gönderimsiz bir bilet formu iskeletiyle bu belgeyi tamamladınız. İkinci ünite de aynı belgenin görünümünü CSS ile kurdunuz: seçiciler ve özgüllük hangi kuralın kazanacağını, kaskad ve kalıtım kurallarının nasıl birleştiğini, kutu modeli (`content/padding/border/margin` ve `box-sizing: border-box`) genişlik hesaplarındaki “gizemi” nasıl çözdüğünü, birimler (`px/rem/%/vw`) hangi durumda hangisinin tercih edileceğini gösterdi. Üçüncü ünite de öğeleri birbirine göre dizmeyi öğrendiniz: Flexbox tek eksen de (nav çubuğu), Grid iki eksen de (kart ızgarası, koltuk planı) yerleşim sağladı — hangisinin ne zaman tercih edileceğini, tek boyutlu ile iki boyutlu problem ayrımı üzerinden gördünüz. Son ünite de tüm bu yapının her ekranda çalışmasını sağladınız: viewport meta etiketi, mobil-önce medya sorguları, akışkan görseller ve erişilebilirlik temelleri (alt metin, etiket-girdi eşleşmesi, kontrast, klavye odağı) ile sayfanız hem bir telefonda hem bir masaüstünde, hem gören hem klavyeyle gezinen bir kullanıcı

cı için düzgün çalışır hale geldi. Bu dört ünite birlikte, Etkinlik/Bilet Sisteminin statik etkinlik sayfasını — nav, etkinlik kartları ızgarası, koltuk planı ızgarası, bilet formu iskeleti, iki kırılım — tamamladı. Gelecek hafta bu *hareketsiz* sayfaya ilk kez davranış ekleyeceksiniz: JavaScript’in dil modelini öğrenip TypeScript’e geçecek, DOM üzerinden koltuklara tıklama, form doğrulama gibi etkileşimler kazandıracaksınız.

Kendinizi Sınavın Cevapları

Soru (Ünite 1 — semantik öğelerin en az iki faydası). Bir sayfanın tamamını `<div>` yerine `<header>/<nav>/ <main>/<footer>` gibi semantik öğelerle yazmanın birden çok somut faydası vardır. Birincisi erişilebilirlik/keşfedilebilirliktir: ekran okuyucular ve arama motorları, sayfanın hangi kısmının menü, hangi kısmının ana içerik ya da altbilgi olduğunu doğrudan semantik etiketlerden anlar — `<div>`’in kendisi hiçbir anlam taşımaz, yalnızca `class` adına bakılarak tahmin yürütülebilir, bu da otomatik araçlar için güvenilir değildir. İkincisi bakım kolaylığıdır: kodu okuyan bir geliştirici, bir `<nav>` etiketini gördüğü an bunun bir gezinme menüsü olduğunu anlar; onlarca aynı görünen `<div>` arasında doğru olanı aramak zorunda kalmaz. Üçüncüsü, bu ünitenin sonunda gördüğünüz gibi, semantik öğeler zaten **bileşen sınırlarını** da işaret eder — Hf.4’te her landmark öğesi ayrı bir React bileşenine dönüşecek, `div` çorbasında bu sınırları sonradan bulmak çok daha zahmetli olurdu.

Soru (Ünite 2 — id ile sınıf seçici çakışması). Bir öğeye hem `.kart { color: mavi; }` hem `#ozel-kart { color: kirmizi; }` kuralı uygulanmışsa **kırmızı** kazanır. Bunun nedeni özgüllüktür: id seçicisi (`#ozel-kart`), sınıf seçicisinden (`.kart`) daha ağır/özgöl sayılır — CSS’in özgüllük sıralamasında id, sınıftan her zaman önde gelir. Kaskad mekanizması çakışan kurallar arasında önce özgüllüğe bakar; dosyadaki yazım sırası yalnızca özgüllük eşit olduğunda devreye girer. Burada özgüllük zaten eşit olmadığı için, hangi kuralın dosyada önce ya da sonra yazıldığına bir önemi yoktur — `#ozel-kart` her koşulda kazanır.

Soru (Ünite 3 — koltuk planında Grid’in Flexbox’a üstünlüğü). Satır × sütun düzenindeki bir koltuk planı için Flexbox yerine Grid tercih edilmelidir, çünkü Flexbox **tek eksenlidir**: öğeler tek bir satırda ya da sütunda dizilir, satırlar sarıldığında (`flex-wrap` ile) sütunların hizası — bir satırdaki 4. koltuğun bir alttaki satırın 4. koltuğuyla tam hizalı olması — garanti edilmez, çünkü her satır kendi içeriğine göre bağımsız şekillenir. Grid ise **iki eksenlidir**: hem satır hem sütun boyutları (`grid-template-rows/grid-template-columns`) önceden birlikte tanımlanır, bu yüzden her hücre hem doğru satırda hem doğru sütunda, komşu satırlarla tam hizalı biçimde yerleşir — koltuk planı, sayfa ızgarası gibi “tablo benzeri” yerleşimler için Grid doğal araçtır.

Soru (Ünite 4 — mobil-önce medya sorguları neden min-width kullanır). Mobil-önce yaklaşımda medya sorguları genellikle `min-width` ile yazılır. Bunun nedeni stratejinin yönüdür: önce en dar/mobil ekran için temel stiller (medya sorgusu dışında, her zaman geçerli kurallar olarak) tanımlanır; ekran genişledikçe `min-width` eşikleri aşıldığında bu temel kurallara **ek** stiller eklenir, hiçbir şey çıkarılmaz. Bu, “önce basit, sonra zenginleştir” mantığına uyar: çoğu mobil kullanıcı

zaten hiçbir zaman geniş-ekran kurallarını indirip işlemez, bu da mobil tarafta gereksiz CSS yükünü önler. `max-width` ile yazılan “masaüstü-önce” yaklaşımda ise tam tersi olurdu — önce geniş ekran için karmaşık kurallar yazılır, sonra dar ekranda bunların bir kısmı iptal edilmeye çalışılırdı; bu hem daha kırılgan hem de mobil kullanıcıya gereksiz kural indirtir.